TEACHING PACKAGE FOR 6CM504 — EXERCISE 6 (Second Assignment)

1. WHAT IS THIS ASSIGNMENT?

"This is the **second assessed exercise** for 6CM504, worth **50%** of your module mark.
You must complete **both Task 1 and Task 2**.
Your submission will contain:

- **task1_username.csp**
- **task2_username.csp**
- **a short accompanying technical report**

This is an *individual* assignment — no sharing, no collaboration."

2. WHAT TO SUBMIT

"You will submit:

✓ One CSP model for Task 1

```
task1_username.csp
```

✓ One CSP model for Task 2

```
task2_username.csp
```

✓ A short report (~1,000–1,500 words) including:

- your interpretation of the algorithms
- how you built the CSP models
- the refinements you tested
- screenshots/output from FDR
- a short reflection
- the final working CSPM models

Your report must include:

- **Task 1**:
  Verification that **Dekker's algorithm** works *when implemented correctly*
  AND
  Demonstration that it **breaks** under certain compiler optimisations.
- **Task 2**:
  Full CSP model of **Peterson's algorithm for 3 processes**,

proof of **mutual exclusion**,
proof of **no blocking of progress**,
and demonstration of **bounded waiting**."

3. WHAT STUDENTS MUST DO IN TASK 1

"Task 1 is about **Dekker's algorithm**, a classical mutual exclusion protocol for **two processes** using only **shared memory**.

The brief says:

- You must have a **working model** of the original Dekker algorithm.
- You must verify **correctness** of the model using CSP **traces refinement**.
- Then, you must **model compiler optimisations** described in the brief:

❗ Optimisation A:

Compiler removes writes to `flag1` or `flag2` if they appear unused.

❗ Optimisation B:

Compiler removes updates to `turn` if unused.

❗ Your model must allow:

- optimisation A on or off
- optimisation B on or off
- both on
- both off

You must then **prove with FDR** that:

✓ Without optimisations → Dekker works
✓ With optimisations → the algorithm can **break**

This is the core learning outcome."

4. WHAT STUDENTS MUST DO IN TASK 2

"Task 2 focuses on **Peterson's algorithm for n processes**, using:

- an array **level[n]**
- an array **last_to_enter[n-1]**

The algorithm ensures:

✓ mutual exclusion
✓ no lockout / bounded waiting
✓ progress
✓ no deadlock

Your job is:

Build a CSP model for n = 3 processes.

Your CSPM must include:

- channel operations modelling reads/writes of `level[i]`
- modelling of `last_to_enter`
- each process moving through levels 0 → 1 → 2
- the inner loop behaviour
- entry into critical section
- setting `level[i] = -1` on exit
- parallel composition of all 3 processes
- appropriate alphabets
- specification process that expresses mutual exclusion
- and a refinement check:

```
assert SYSTEM [T= SPEC]
```

You must also show:

- the system is **deadlock-free**
- the system ensures **bounded waiting**
- progress is ensured

This is a more complex modelling task than Task 1."

LECTURE 12

LECTURE 12 CSP BASICS — WHAT STUDENTS MUST UNDERSTAND BEFORE MODELLING**

This section is **what you explain to prepare students to write CSP models**.

2.1 Processes & Events

"In CSP, everything is a **process** performing **events**.

Example:

```
P = a -> b -> P
```

P first performs `a`, then `b`, then loops."

2.2 Prefixing

Prefixing defines sequence:

`a -> P` means "do event a, then behave like P".

2.3 Channels and Data

Lecture 12 explains data passing:

```
channel write : Int
Process = write?x -> Process
```

`x` is bound at runtime — this is essential for modelling variables like `flag1`, `turn`, `level[i]`."

2.4 Choice (external and internal)

External choice:

```
P = a -> P [] b -> P
```

Internal choice (non-deterministic):

```
P = a -> P |~| b -> P
```

2.5 Parallel Composition

Parallel composition is the key to modelling mutual exclusion.

```
System = P1 [| {|sharedEvents|} |] P2
```

Processes synchronise on shared events.

To model Dekker or Peterson, you must define:

- shared memory updates as events
- shared memory reads as events
- synchronisation where appropriate
- interleaving of independent events

2.6 Hiding and Renaming

Used for abstractions.

Dekker rarely requires renaming; Peterson sometimes does, especially for internal events.

2.7 Refinement

Refinement expresses **correctness**.

If `Spec` is a specification process and `Impl` is your model:

```
assert Impl [T= Spec]
```

This means:

“All traces of Impl must be allowed by Spec.”

This is how you prove mutual exclusion:

```
Spec = (cs1 -> STOP) [] (cs2 -> STOP)
```

But **never both** in parallel.

2.8 Deadlock / Livelock

Dekker can deadlock if compiler optimisations break assumptions.
Peterson should not deadlock.

Check:

```
assert System :[deadlock free]
assert System :[divergence free]
```

Step by step modeling

STEP-BY-STEP MODELLING OF TASK 1 (DEKKER)**

Now we build **exact CSPM code** step by step (spoken explanation).

3.1 Step 1 — Declare channels

You need channels for:

- setting flag1, flag2
- reading flag1, flag2
- setting turn
- reading turn
- entering critical section
- leaving critical section

Example:

```
channel setflag1, setflag2 : Bool
channel setturn : Int
channel enter1, enter2, exit1, exit2
```

3.2 Step 2 — Model shared memory

Model the flag array as a process:

```
Flag1(value) =
    setflag1?x -> Flag1(x)
  [] readflag1!value -> Flag1(value)
```

Do similarly for:

- Flag2
- Turn

3.3 Step 3 — Model Dekker process

Each process alternates between:

- setting its flag to true
- waiting until other flag is false OR it is its turn
- entering critical section
- setting flag to false
- repeating

Simplified CSP:

```
P1 =
    setflag1!true ->
    wait1

wait1 =
    readflag2?f2 ->
    if f2 == false then enter1 -> exit1 -> setflag1!false -> P1
    else readturn?t ->
         if t == 1 then wait1
         else wait1
```

This is simplified but teachable.

3.4 Step 4 — Add Compiler Optimisations

Optimization A: flag writes removed
Optimization B: turn writes removed

Model them as conditional choices:

```
P1 =
    ( optimisationA -> SKIP [] notOptimisationA -> setflag1!true )
    ->
    wait1
```

Optimisation flags:

```
channel optA, optB : Bool
```

3.5 Step 5 — Compose whole system

```
System =
    P1
    [| {|shared memory events|} |]
    P2
    [| {|shared memory events|} |]
    Memory
```

3.6 Step 6 — Write the safety specification

“Both processes must **never** enter critical section simultaneously.”

```
SPEC =
    enter1 -> exit1 -> SPEC
  [] enter2 -> exit2 -> SPEC
```

3.7 Step 7 — Refinement checks

```
assert System [T= SPEC]
assert System :[deadlock free]
```

3.8 Step 8 — Show breakage under optimisation

When optimisations applied → FDR produces a counterexample trace.

Students must copy + explain this trace in their report:

Example:

```
enter1, enter2
```

Impossible under correct mutual exclusion → therefore Dekker fails.

STEP-BY-STEP MODELLING OF TASK 2

STEP-BY-STEP MODELLING OF TASK 2 (PETERSON'S ALGORITHM FOR 3 PROCESSES)**

4.1 Step 1 — Define channels

```
channel setlevel, readlevel : Int.Int
channel setlast, readlast : Int.Int
channel cs : Int
```

where `cs!i` means "process i entered critical section".

4.2 Step 2 — Model the shared arrays

level array:

```
Level(i, value) =
    setlevel.i?x -> Level(i, x)
  [] readlevel.i!value -> Level(i, value)
```

last_to_enter array:

```
Last(l, value) =
    setlast.l?x -> Last(l, x)
  [] readlast.l!value -> Last(l, value)
```

4.3 Step 3 — Process behaviour

Each process:

- moves level from 0 → 1 → 2
- at each level sets `last_to_enter[l] = i`
- waits until safe
- enters critical section
- sets level[i] = -1
- recurses

4.4 Step 4 — Compose 3 processes in parallel

```
SYSTEM =
    Process(0)
    [| Shared |]
    Process(1)
    [| Shared |]
    Process(2)
    [| Shared |]
    Memory
```

4.5 Step 5 — Specification for mutual exclusion

```
SPEC =
    cs?i -> cs?j -> STOP  -- but forbidden if i != j
```

More accurate:

```
SPEC =
    (cs.0 -> cs.0 -> SPEC)
  [] (cs.1 -> cs.1 -> SPEC)
  [] (cs.2 -> cs.2 -> SPEC)
```

4.6 Step 6 — Refinement

```
assert SYSTEM [T= SPEC]
assert SYSTEM :[deadlock free]
assert SYSTEM :[divergence free]
```

0) What to tell students they must submit

"For Assessment 2 you must submit **one or two CSP scripts** and **supporting documentation**. Specifically, the brief says your submission consists of **CSP source scripts** (`task1_username.csp` and `task2_username.csp`, as appropriate) and any accompanying documentation, such as a short report."

Then:

"Your report must show:

1. your model design decisions (events, processes, channels),
2. the **assertions** you checked in **FDR**,
3. the **results** (pass/fail) and what the counterexample means when something fails,
4. and how your model matches the problem statement."

FULLY WORKED MODEL SOLUTION — Task 1 (Dekker + compiler optimisations)

Save as: `task1_username.csp`

```
-- ============================================================
-- 6CM504 Exercise 6 - Task 1
-- Dekker's algorithm + compiler optimisation variants
-- ============================================================

-- Observable critical section events
channel enter, exit : {1,2}

-- Shared memory events for flags and turn
channel wflag : {1,2}.Bool
channel rflag : {1,2}.Bool

channel wturn : {1,2}        -- write turn = 1 or 2
channel rturn : {1,2}        -- read turn returns 1 or 2

-- ------------------------------------------------------------
-- Shared variable processes (single-writer/multi-reader style)
-- We model each flag[i] as a state-holding process.
-- ------------------------------------------------------------
```

```
FLAG(i, val) =
    wflag.i?x -> FLAG(i, x)
  [] rflag.i!val -> FLAG(i, val)

TURN(val) =
    wturn?x -> TURN(x)
  [] rturn!val -> TURN(val)

-- ------------------------------------------------------------
-- Helper: other process id
-- ------------------------------------------------------------
other(1) = 2
other(2) = 1

-- ------------------------------------------------------------
-- Dekker process with optimisation toggles
-- optFlag: if true, compiler removes "yield" flag writes inside
contention handling
-- optTurn: if true, compiler removes the turn update on exit
-- ------------------------------------------------------------

P(i, optFlag, optTurn) =
    -- wants to enter: set flag[i] = true
    wflag.i!true ->
    TRY(i, optFlag, optTurn)

TRY(i, optFlag, optTurn) =
    -- read other flag
    rflag.(other(i))?f ->
    if f == false then
        -- safe to enter
        enter.i -> exit.i ->
        -- on exit: set turn to other (unless optimised away)
        (if optTurn then SKIP else wturn.(other(i)) -> SKIP) ;
        -- set my flag false
        wflag.i!false ->
        P(i, optFlag, optTurn)
    else
        -- contention: both want to enter, so check turn
        rturn?t ->
        if t == other(i) then
            -- yield: set my flag false, wait until turn changes,
then set true again
            (if optFlag then SKIP else wflag.i!false -> SKIP) ;
            WAIT_TURN(i, optFlag, optTurn)
        else
            -- my turn, keep trying
            TRY(i, optFlag, optTurn)

WAIT_TURN(i, optFlag, optTurn) =
    rturn?t ->
    if t == other(i) then
        WAIT_TURN(i, optFlag, optTurn)
    else
        -- re-raise intent to enter
        (if optFlag then SKIP else wflag.i!true -> SKIP) ;
        TRY(i, optFlag, optTurn)

-- ------------------------------------------------------------
-- Whole system for a given optimisation setting
```

```
-- Initial values: flag1=false, flag2=false, turn=1 (arbitrary)
-- ------------------------------------------------------------

SYSTEM(optFlag, optTurn) =
    (P(1, optFlag, optTurn) [| {|wflag.1, rflag.1, wflag.2, rflag.2,
wturn, rturn|} |] P(2, optFlag, optTurn))
    [| {|wflag.1, rflag.1, wflag.2, rflag.2, wturn, rturn|} |]
    (FLAG(1,false) [|{|wflag.2, rflag.2, wturn, rturn|}|]
FLAG(2,false) [|{|wturn, rturn|}|] TURN(1))

-- ------------------------------------------------------------
-- Safety specification: mutual exclusion over enter/exit
-- "Never see enter.1 while 2 is inside, and never see enter.2 while
1 is inside"
-- In traces terms, enter.1 must be followed by exit.1 before any
enter.2, and vice versa.
-- ------------------------------------------------------------

SPEC_CS =
    enter.1 -> exit.1 -> SPEC_CS
  [] enter.2 -> exit.2 -> SPEC_CS

-- ------------------------------------------------------------
-- Assertions for the assignment
-- Task requirement: prove correct model using traces refinement.
-- Then show breakage under optimisations.
-- ------------------------------------------------------------

-- Baseline: should PASS (correct Dekker)
assert SYSTEM(false, false) [T= SPEC_CS]
assert SYSTEM(false, false) :[deadlock free]
assert SYSTEM(false, false) :[divergence free]

-- Optimised variants: expected to FAIL mutual exclusion (at least
one should)
assert SYSTEM(true,  false) [T= SPEC_CS]
assert SYSTEM(false, true ) [T= SPEC_CS]
assert SYSTEM(true,  true ) [T= SPEC_CS]
```

1) What the assignment requires for Task 1 (in plain terms)

From the brief, Task 1 requires you to:

6CM504 - Exercise 6-Second-assi…

1. Build a CSP model of **Dekker's algorithm** (correct version).
2. Verify correctness using a **safety property modelled as traces refinement**.
3. Implement the compiler optimisation(s) described in the brief (write removal effects).
4. Re-run the same safety check to show the algorithm **can break** under those optimisations.
5. Provide your CSP file + a short report explaining the model and the FDR evidence.

So your testing must show:

- **Baseline passes** the traces refinement safety check
- **At least one optimised variant fails** and you present the counterexample trace as evidence

---

2) Exactly how to test it in FDR (step-by-step)

Step 0 — Prepare the CSPM file

Ensure your file contains the assertions exactly like:

- `assert SYSTEM(false,false) [T= SPEC_CS`
- `assert SYSTEM(true,false) [T= SPEC_CS`
- `assert SYSTEM(false,true) [T= SPEC_CS`
- `assert SYSTEM(true,true) [T= SPEC_CS`

and optionally:

- `assert SYSTEM(false,false) :[deadlock free]`
- `assert SYSTEM(false,false) :[divergence free]`

(You already have these.)

Step 1 — Load the model

1. Open **FDR**
2. `File → Open…`
3. Select your `task1_username.cspm`
4. Wait until it finishes parsing/compiling (no syntax errors)

Step 2 — Find the Assertions list

In the left panel, you will see a tree with sections like:

- **Processes**
- **Assertions** (or "Checks")

Click **Assertions**.

You should see each `assert ...` line listed as a clickable item.

Step 3 — Run the baseline correctness test (required)

Run:

✅ **Baseline mutual exclusion check**

- Click: `assert SYSTEM(false,false) [T= SPEC_CS`
- Right-click → **Check** (or press Run)

Expected result: PASS.

This is the assignment's core requirement: "prove correct using traces refinement safety property."

6CM504 - Exercise 6-Second-assi…

Step 4 — Run the optimisation variants (required)

Now run each:

- `assert SYSTEM(true,false) [T= SPEC_CS`
- `assert SYSTEM(false,true) [T= SPEC_CS`
- `assert SYSTEM(true,true) [T= SPEC_CS`

Expected result: At least one should FAIL.

Step 5 — View the counterexample trace (required evidence)

When an assertion FAILS:

1. Click the failed assertion
2. Open **Counterexample / Witness / Trace** (wording depends on FDR version)
3. You will see a **sequence of events** like:

```
enter.1, enter.2, ...
```

Your job is to identify the key violation:

✅ **Violation pattern:**

- `enter.1` occurs
- before `exit.1` occurs, you see `enter.2`
  (or the opposite order)

That's a direct mutual exclusion violation.

Step 6 — Optional but strong tests (recommended)

Run:

- `assert SYSTEM(false,false) :[deadlock free]`
- `assert SYSTEM(false,false) :[divergence free]`

These aren't explicitly demanded for Task 1, but they make your report stronger and help explain progress/livelock issues in optimisation cases.

---

3) What to include in your submission/report for Task 1 (exact checklist)

A) In the CSP file

Your CSPM must include:

- shared memory model (`FLAG`, `TURN`)
- algorithm processes (`P`, `TRY`, `WAIT_TURN`)
- system composition (`SYSTEM(...)`)
- safety specification (`SPEC_CS`)
- required traces refinement assertions (baseline + optimisation variants)

6CM504 - Exercise 6-Second-assi…

B) In the report: what to write (template text)

1) State what you verified (baseline)

Write something like:

"We modelled Dekker's mutual exclusion algorithm for two processes. We specified mutual exclusion as a safety property (`SPEC_CS`) and verified correctness using **traces refinement** in FDR. The baseline model `SYSTEM(false,false)` satisfied the property."

Include:

- a screenshot showing PASS for `assert SYSTEM(false,false) [T= SPEC_CS`

2) Explain the safety property (SPEC_CS)

Write:

"The safety specification `SPEC_CS` constrains observable behaviour so that an `enter.i` event must be followed by `exit.i` before any other process can perform `enter.j`. Therefore any trace containing `enter.1` followed by `enter.2` without an intervening `exit.1` violates mutual exclusion."

Include:

- the `SPEC_CS` snippet (small code excerpt)

3) Show compiler optimisation breakage (required)

Write:

"We then modelled the compiler optimisations described in the brief by allowing certain writes to be skipped (controlled by `optFlag` and `optTurn`). Under these settings, at least one traces refinement check failed, and FDR produced a counterexample trace demonstrating a mutual exclusion violation."

Include:

- screenshot showing FAIL for the chosen variant(s)
- paste the counterexample trace (or the key part)

4) Interpret the counterexample trace (required)

Write:

"The counterexample shows both processes entering the critical section without the first leaving, e.g. `enter.1` occurs and before `exit.1`, the trace contains `enter.2`. This violates the mutual exclusion safety property, demonstrating that removing key shared-memory writes can break Dekker's correctness."

5) (Optional) Add deadlock/divergence evidence

If you ran those checks:

"Additionally, we checked deadlock-freedom and divergence-freedom for the baseline model. These checks support that the model is well-formed and does not trivially block or livelock in the correct (unoptimised) configuration."

Include screenshots if you want.

---

4) Minimal evidence pack (what markers want to see)

To be safe, include these screenshots in the report:

1. ✓ PASS: `SYSTEM(false,false) [T= SPEC_CS`
2. ✓ FAIL: one optimisation variant `[T= SPEC_CS`
3. ✓ Counterexample trace view showing the violation
4. (Optional) ✓ PASS: deadlock free and divergence free for baseline

---

5) If you tell me which variant failed, I'll write the exact "results section" paragraph for you

Reply with:

- Which assertion failed (`SYSTEM(true,false)` or `SYSTEM(false,true)` or `SYSTEM(true,true)`)
- And paste the first ~10 events of the counterexample trace

…and I'll produce the exact report wording ("Results & Discussion") matching that trace.